

Paper Summaries & Viva Q&A

Labyrinth Breach – AI641 | 6 Must-Know Papers

1. PPO (Schulman et al., 2017)

Title: Proximal Policy Optimization Algorithms

Link: <https://arxiv.org/abs/1707.06347>

Problem

Policy gradient methods like TRPO are effective but complex (require conjugate gradient, line search). Need a simpler method that still prevents destructively large policy updates.

Key Idea

Instead of a hard trust-region constraint (TRPO), PPO uses a **clipped surrogate objective** that automatically limits how much the policy can change per update.

Core Equation

$$L^{CLIP} = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the probability ratio.

What this means:

- If $r_t > 1+\epsilon$: policy moved too far in the “good” direction → clipped, no extra gradient
- If $r_t < 1-\epsilon$: policy moved too far in the “bad” direction → clipped
- The $\min$ ensures we take the more pessimistic (conservative) estimate

Other Components

- **GAE (λ):** Generalized Advantage Estimation. Blends TD (low variance, high bias) with Monte Carlo (high variance, low bias). $\lambda = 0.95$ favors lower variance.

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

- **Entropy bonus (β):** Added to loss to encourage exploration. Prevents premature convergence to deterministic policy.
- **Value function loss:** Trains the critic $V(s)$ alongside the policy.
- **Total loss:** $L = L^{CLIP} - c_1 L^{VF} + c_2 H[\pi]$

Connection to Labyrinth Breach

- We use PPO as our training algorithm with $\epsilon = 0.2$, $\lambda = 0.95$, $\beta = 0.005$
- Parameter sharing: all 3 Sentinels share one PPO brain, both Runners share another
- Continuous action space (2 floats: moveX, moveZ) – PPO handles this natively
- Unity ML-Agents implements PPO with linear LR decay and configurable entropy

Viva Q&A

Q: What does clipping do in PPO?

A: It prevents the policy from changing too much in one update. If the ratio r_t goes outside $[1-\epsilon, 1+\epsilon]$, the gradient is zeroed out. This gives stability similar to TRPO but is much simpler to implement.

Q: What is the advantage function?

A: It measures how much better an action is compared

to the average. $A(s, a) = Q(s, a) - V(s)$. Positive advantage means the action was better than expected, negative means worse.

Q: What happens if ϵ is too large?

A: Large ϵ allows bigger policy updates per step. This can cause training instability because the policy may jump to a bad region and never recover. Typical value is 0.1–0.3.

Q: What happens if ϵ is too small?

A: Learning becomes very slow because each update barely changes the policy. It would take many more iterations to converge.

Q: Why $\gamma = 0.99$ and not 0.9?

A: A capture event may happen 50–100 steps after a good positioning move. With $\gamma = 0.9$, reward from 50 steps ahead is discounted to $0.9^{50} = 0.005$ (almost zero). With $\gamma = 0.99$, it is $0.99^{50} = 0.605$ (still meaningful). So 0.99 lets the agent learn from delayed outcomes.

Q: What is the difference between PPO and TRPO?

A: TRPO uses a hard KL-divergence constraint (requires second-order optimization). PPO approximates this with a simple clip. PPO is easier to implement, parallelizes better, and works comparably well.

2. MADDPG (Lowe et al., 2017)

Title: Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments

Link: <https://papers.nips.cc/paper/7217>

Problem

Independent learning in multi-agent settings is unstable because each agent's environment is non-stationary (other agents are changing). Q-learning fails because the Markov property breaks.

Key Idea: CTDE

Centralized Training, Decentralized Execution:

- **Training:** Each agent has its own critic that sees *all* agents' observations and actions: $Q_i(x, a_1, \dots, a_N)$
- **Execution:** Each agent uses only its own actor $\pi_i(o_i)$ with local observations
- This solves non-stationarity because the critic can model other agents' behavior

Method

- Extension of DDPG (Deep Deterministic Policy Gradient) to multi-agent
- Each agent i has: actor $\mu_i(o_i)$, critic $Q_i(x, a_1, \dots, a_N)$
- Critic gradient: $\nabla_{\theta_i} J = \mathbb{E}[\nabla_{\theta_i} \mu_i \cdot \nabla_{a_i} Q_i]$
- Uses experience replay and target networks (like DQN)

Connection to Labyrinth Breach

- We use the CTDE *principle* but not the MADDPG *algorithm*
- Our PPO + parameter sharing is a simpler form: shared weights serve as implicit centralization
- MADDPG uses deterministic policies; we use stochastic (PPO outputs a distribution)
- MADDPG requires one critic per agent; our approach

uses one shared value function per team

Viva Q&A

Q: Why didn't you use MADDPG?

A: MADDPG requires one separate critic per agent and uses off-policy replay buffers. PPO with parameter sharing is simpler, on-policy, and natively supported by Unity ML-Agents. MAPPO (Yu 2022) showed PPO matches or beats MADDPG in cooperative settings.

Q: How is CTDE different from parameter sharing?

A: In CTDE, each agent has its own network but the critic sees global state during training. In parameter sharing, all same-role agents literally share one network. Parameter sharing is more restrictive (agents cannot specialize) but more sample-efficient (3x more data per network update).

Q: What is non-stationarity in MARL?

A: From one agent's perspective, the environment keeps changing because other agents are updating their policies. This violates the Markov assumption. Solutions: centralized critic (MADDPG), shared network (our approach), or communication.

3. MAPPO (Yu et al., 2022)

Title: The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games

Link: https://proceedings.neurips.cc/paper_files/paper/2022/hash/9c1535a02f0ce079433344e14d91511.html

Problem

Complex MARL algorithms (QMIX, MADDPG, MAPPO variants) are assumed necessary for cooperative tasks. Is simple PPO actually competitive when implemented carefully?

Key Finding

Yes. With proper engineering choices, PPO matches or beats specialized MARL algorithms on SMAC, MPE, and Hanabi benchmarks.

Critical Implementation Tricks

1. **Value normalization:** Normalize returns by running mean/std
2. **Input normalization:** Normalize observations
3. **Orthogonal initialization:** Initialize network weights with orthogonal matrices
4. **Gradient clipping:** Clip gradient norms to 10
5. **Large batch sizes:** Use large buffers relative to environment complexity
6. **Proper entropy tuning:** Start with enough entropy, let it decay
7. **Death masking:** Handle terminated agents properly

Connection to Labyrinth Breach

- Directly validates our choice: "PPO is not a weak baseline, it is a strong one"
- Our hyperparameters follow MAPPO recommendations: large buffer (10240), batch 1024, 3 epochs, LR 3×10^{-4}
- We add parameter sharing (all Sentinels share weights), which MAPPO also evaluates
- Our entropy $\beta = 0.005$ follows their guidance on exploration

Viva Q&A

Q: What makes your PPO implementation "careful"?

A: We follow MAPPO's recipe: proper batch/buffer ratio (1024/10240), linear LR decay, entropy bonus 0.005, 3 epochs per update, gradient clipping, and 256-unit 2-layer network. These are not arbitrary – they come from MAPPO's ablation study.

Q: Doesn't parameter sharing prevent agent specialization?

A: Yes, but each agent still receives different observations (different positions, ray hits). The shared network learns a *role-conditioned* policy: given the current observation, what should a Sentinel do? Agents behave differently because they see different things, not because they have different weights.

Q: How does MAPPO differ from independent PPO?

A: MAPPO can use a centralized value function (critic sees global state). Independent PPO uses only local observations for both actor and critic. We use a variant closer to independent PPO with parameter sharing.

4. QMIX (Rashid et al., 2018)

Title: QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent RL

Link: <https://arxiv.org/abs/1803.11485>

Problem

In cooperative MARL, how do you learn a joint team value function Q_{tot} that decomposes into per-agent values Q_i while still allowing decentralized execution?

Key Idea: Monotonic Mixing

- Each agent i learns its own $Q_i(o_i, a_i)$ using only local observations
- A **mixing network** combines them: $Q_{tot} = f(Q_1, Q_2, \dots, Q_N)$
- **Monotonicity constraint:** $\frac{\partial Q_{tot}}{\partial Q_i} \geq 0$ for all i
- This ensures: $\arg \max_{a_i} Q_i = \arg \max_{a_i} Q_{tot}$ (decentralized argmax = joint argmax)
- Mixing network weights are produced by hypernetworks conditioned on global state

Why Monotonicity?

If Q_{tot} increases whenever any Q_i increases, then each agent can independently pick its best action and the team action is guaranteed to be jointly optimal. No coordination needed at execution time.

Connection to Labyrinth Breach

- QMIX is for **discrete** actions (argmax over Q-values). Our agents use **continuous** actions (2D movement). QMIX cannot directly apply.
- QMIX uses value decomposition; we use policy gradient (PPO). Different paradigm.
- Both address the same goal: team coordination under shared reward
- QMIX's credit assignment is implicit (through mixing weights). Ours is explicit (reward shaping per agent).

Viva Q&A

Q: Why PPO instead of QMIX?

A: QMIX requires discrete action spaces (it does argmax

over Q-values). Our agents output 2 continuous floats for 2D movement. PPO naturally handles continuous actions by outputting a Gaussian distribution. Converting to discrete would lose movement precision.

Q: What is value decomposition?

A: Breaking the team's joint value $Q_{tot}(s, a_1, \dots, a_N)$ into per-agent values $Q_i(o_i, a_i)$. The challenge is ensuring the decomposition is consistent: agents picking locally optimal actions should also be globally optimal.

Q: What is the limitation of QMIX's monotonicity?

A: It cannot represent non-monotonic interactions. Example: if agent A's action is good only when agent B does a specific action. QTRAN and QPLEX address this, but at higher complexity.

5. COMA (Foerster et al., 2018)

Title: Counterfactual Multi-Agent Policy Gradients

Link: <https://ojs.aaai.org/index.php/AAAI/article/view/11794>

Problem

In cooperative MARL with shared team reward, how does each agent know whether *its specific action* helped or hurt? This is the **credit assignment problem**.

Key Idea: Counterfactual Baseline

- Standard baseline: $V(s)$ (average value of the state)
- COMA baseline: "What reward would the team get if I had taken a *different* action while everyone else did the same thing?"
- Advantage for agent i :

$$A_i(s, a_i) = Q(s, \mathbf{a}) - \sum_{a'_i} \pi_i(a'_i | o_i) \cdot Q(s, (a'_i, \mathbf{a}_{-i}))$$

- This marginalizes over agent i 's actions while keeping others fixed
- Uses a centralized critic that can compute Q for any action combination

Why It Matters

Consider: Sentinel A blocked a corridor. Sentinel B tagged the Runner. Both get +1.0 team reward. But who "deserves" the credit? COMA would ask: "If Sentinel A had NOT blocked, would the capture still happen?" If not, Sentinel A gets high advantage.

Connection to Labyrinth Breach

- We do NOT use COMA. We use reward shaping instead.
- Our approach: directly reward coordination events (pincer = +bonus, corridor block = +bonus) via `TrapEventDetector`
- **Tradeoff:** COMA is principled but requires a centralized critic and discrete actions. Our shaping is manual but works with continuous actions and PPO.
- Future work: compare PPO+shaping vs MA-POCA (Unity's credit assignment method)

Viva Q&A

Q: How do you handle credit assignment?

A: We use tactical reward shaping instead of counterfactual methods. Our `TrapEventDetector` detects coordination events (pincer formations, corridor blocks, exit denial)

and gives individual bonuses to the contributing agents. This is more manual than COMA but compatible with our continuous-action PPO setup.

Q: What is the advantage of COMA over reward shaping?

A: COMA automatically determines who contributed. Reward shaping requires us to manually define what counts as a contribution. COMA is more general, but requires discrete actions and a centralized critic that can evaluate counterfactual action combinations.

Q: What is MA-POCA and how does it relate to COMA?

A: MA-POCA (Unity ML-Agents) uses a centralized critic like COMA but adds "posthumous" credit – agents that are removed mid-episode (e.g., captured Runners) still receive gradient updates. We plan to compare PPO vs MA-POCA in future work.

6. ViPER (Wang et al., 2025)

Title: ViPER: Visibility-based Pursuit-Evasion via Reinforcement Learning

Link: <https://proceedings.mlr.press/v270/wang25k.html>

Problem

Standard pursuit-evasion assumes full observability. Real robots have limited sensing. How do you train pursuit policies when agents can only see what is in their line of sight?

Key Idea

- Formulates PE as a visibility-constrained problem: pursuers must *find* targets, not just chase them
- Uses structured observations based on visibility polygons and search frontiers
- Trains team pursuit policies with RL that learn to coordinate search patterns
- Evaluates on maps with occlusion, walls, and limited field of view

What They Do That We Also Do

- Line-of-sight constraints (agents cannot see through walls)
- Memory of last-known positions when targets leave FOV
- Coordinated pursuit requiring spatial awareness
- Evaluation on maps with varying complexity

What We Add Beyond ViPER

- **Asymmetric teams:** 3v2 (ViPER typically uses equal numbers)
- **Dynamic topology:** Walls shift during episodes (ViPER uses static maps)
- **Exit objectives:** Runners can win by reaching exits (not just survival)
- **Tactical reward shaping:** Explicit coordination bonuses
- **Curriculum:** 4-stage difficulty progression

Viva Q&A

Q: How does your work differ from ViPER?

A: ViPER focuses on visibility-based search in static environments with equal team sizes. We add three com-

plexity dimensions: dynamic wall shifts that change maze topology during play, asymmetric 3v2 team structure, and exit-based Runner objectives that create a richer strategic space beyond pure survival. We also use a 4-stage curriculum and tactical reward shaping for coordination.

Q: Why not use visibility polygons like ViPER?

A: ViPER operates in 2D with exact visibility polygon computation. We operate in 3D Unity with physics-based ray-casting, which naturally gives approximate visibility. Our 14/16 ray approach is computationally cheaper and integrates directly with Unity ML-Agents.

Q: What is the closest existing work to your project?

A: ViPER for visibility-constrained pursuit, and Matrix-World for the benchmark platform idea. Our contribution is combining dynamic topology, asymmetric teams, exit objectives, and tactical reward shaping into one integrated environment.

Quick Cross-Reference: Paper ↔ Our Design

Paper	What we took / why we differ
PPO	Our training algorithm. Clipped objective, GAE, entropy bonus.
MADDPG	CTDE principle. We use parameter sharing instead of per-agent critics.
MAPPO	Validates PPO for MARL. Our hyperparameters follow their recipe.
QMIX	Shows value decomposition approach. We use policy gradient because our actions are continuous.
COMA	Credit assignment via counterfactual. We use reward shaping via TrapEventDetector instead.
ViPER	Closest PE paper. We add dynamic walls, 3v2, exits, curriculum.